

ChatGPT - DSA swe



Lecture 20 (1).pdf
PDF

read the pdf then explain me the formula for linear probing from it and write a pseudo code for it

Formula for Linear Probing (from the PDF)

Linear probing is a collision resolution method in open addressing hash tables. If a collision occurs at a bin k , linear probing searches forward one bin at a time until it finds an empty one.

From Slide 34 (Page 34 in PDF):

The formula used for linear probing is:

```
ini  
  
bin = (initial + k) % M;
```

Where:

- $\text{initial} = \text{hash_M}(x.\text{hash}(), M)$ → the initial bin where the object x hashes to.
- k = number of attempts (0, 1, 2, ...) for probing.
- M = total number of bins (size of the hash table).
- $\% M$ ensures the bin index wraps around the end of the array.

Pseudo Code for Linear Probing (Insert + Search)

Insert(x)

```
plaintext  
  
function insert(x):  
    initial_bin = hash_M(x, M)  
    for k from 0 to M-1:  
        bin = (initial_bin + k) % M  
        if table[bin] is EMPTY or table[bin] is DELETED:
```

```
        table[bin] = x
    return
    raise Exception("Hash table is full")
```

Search(x)

plaintext

```
function search(x):
    initial_bin = hash_M(x, M)
    for k from 0 to M-1:
        bin = (initial_bin + k) % M
        if table[bin] == x:
            return True
        if table[bin] is EMPTY:
            return False
    return False // Table is full and item not found
```

Let me know if you also want the **delete()** operation in linear probing or visual examples.

read the pdf then explain me the formula for quadratic probing from it and everything about it from thr pdf and write a pseudo code for it

Based on the PDF ([Lecture 20 \(1\).pdf](#)), here's a full explanation of **Quadratic Probing**:

What is Quadratic Probing?

Quadratic probing is an **open addressing** collision resolution strategy in hash tables. When a collision occurs at bin k , instead of checking bins linearly ($k+1$, $k+2$, etc.), it checks bins using a **quadratic function of the probe number**.

This helps **reduce clustering** (compared to linear probing), especially **primary clustering**.

Formula for Quadratic Probing (from PDF - Page 35)

The standard formula for quadratic probing is:

```
cpp

bin = (initial + k*k) % M;
```

Where:

- `initial = hash_M(x.hash(), M)` → the base index where `x` initially hashes to.
- `k` = the probe number (starting from 0).
- `M` = size of the hash table.

Each probe checks positions like:

`initial + 02`, `initial + 12`, `initial + 22`, ... until an empty bin is found or the table is fully traversed.

! Problem (mentioned in PDF)

Will `initial + k2` step through all bins?

Not always. Depending on `M` and the quadratic function, not all bins may be reached. That's why this method is **not guaranteed** to probe every slot unless special care is taken.

⚙ Generalized Formula (Page 36 & 37)

A more general formula:

```
cpp

bin = (initial + c1*k + c2*k*k) % M;
```

- If `M = 2m` (a power of 2), then using `c1 = c2 = ½` guarantees full probing.

Optimized version (for powers of 2):

```
cpp
```

```
bin = (initial + (k + k*k)/2) % M;
```

This ensures even growth and complete probing across the table.

Another simple equivalent method (Page 38):

```
cpp
```

```
bin = (bin + k) % M;
```

Pseudo Code for Quadratic Probing

Insert(x)

```
plaintext
```

```
function insert(x):
    initial_bin = hash_M(x, M)
    for k from 0 to M-1:
        bin = (initial_bin + k*k) % M
        if table[bin] is EMPTY or table[bin] is DELETED:
            table[bin] = x
            return
    raise Exception("Hash table is full")
```

Search(x)

```
plaintext
```

```
function search(x):
    initial_bin = hash_M(x, M)
    for k from 0 to M-1:
        bin = (initial_bin + k*k) % M
        if table[bin] == x:
            return True
        if table[bin] is EMPTY:
            return False
    return False // Not found
```

Key Points Recap

- Reduces primary clustering (better than linear probing).
- May still suffer from **secondary clustering**.
- To guarantee complete probing, **M** should ideally be a **power of 2**, and specific **c1**, **c2** values used.
- Slightly more complex but better average performance under moderate load factors.

Let me know if you'd like visual examples or implementation in C++/Java/Python!

read the pdf then explain me the formula for double hashing from it and everything about it in details from the pdf and write a pseudo code for it

Here's a full explanation of **Double Hashing** based on the content from your uploaded PDF "**Lecture 20 (1).pdf**" (Slides 39–52):

What is Double Hashing?

Double Hashing is another **open addressing** technique used for collision resolution in hash tables. Unlike **linear** or **quadratic probing**, it uses a **second hash function** to calculate the *step size* (i.e., how many bins to skip) upon collision.

This helps reduce **both primary and secondary clustering**, making it **more efficient** than linear or quadratic probing when load factor is high.

Formula for Double Hashing (Page 40)

cpp

```
bin = (initial + k * jump) % M
```

Where:

- $\text{initial} = \text{hash1}(x) \% M \rightarrow$ first hash function gives the starting bin
 - $\text{jump} = \text{hash2}(x) \% M \rightarrow$ second hash function gives the step size
 - k = the number of probes (0, 1, 2, ...)
 - M = size of the hash table
-

Important Conditions (Page 41)

To ensure **complete coverage** of the table:

- The **jump size and table size (M)** must be **coprime** (no common divisors).
 - This can be ensured by:
 - Choosing M as a **prime number**, or
 - Restricting the values of the second hash function appropriately.
-

Example (Page 42-51)

For hexadecimal numbers like $5BA$, the digits are used as:

- **Last digit (A)** $\rightarrow$ used for primary hash $\rightarrow$ **initial bin**
- **Second-last digit (B)** $\rightarrow$ used for secondary hash $\rightarrow$ **jump size**

If the jump digit is **odd**, it is directly used as jump size.

If **even**, it is made odd by adding 1.

Example:

For $5BA$:

- $A \rightarrow$ Bin 10 (initial bin)
 - $B \rightarrow$ Jump = $B = 11$ (odd, so used as is)
 - If bin 10 is full, next try = $(10 + 1 \times 11) \% 16 = 5$, then $(10 + 2 \times 11) \% 16 = 0$, etc.
-



Advantages of Double Hashing

- Avoids **primary** and **secondary clustering**
 - Better **distribution** of keys
 - More uniform use of hash table slots
-



Pseudo Code for Double Hashing

Insert(x)

plaintext

```
function insert(x):
    initial = hash1(x) % M
    jump = hash2(x) % M
    if jump == 0:
        jump = 1 // ensure non-zero jump
    for k from 0 to M-1:
        bin = (initial + k * jump) % M
        if table[bin] is EMPTY or table[bin] is DELETED:
            table[bin] = x
            return
    raise Exception("Hash table is full")
```

Search(x)

plaintext

```
function search(x):
    initial = hash1(x) % M
    jump = hash2(x) % M
    if jump == 0:
        jump = 1
    for k from 0 to M-1:
        bin = (initial + k * jump) % M
        if table[bin] == x:
```

```
        return True
    if table[bin] is EMPTY:
        return False
    return False
```

✓ Summary of Double Hashing (from PDF)

Feature	Description
Strategy	Two hash functions: one for bin, one for jump
Goal	Minimize clustering
Hash Table Size	Ideally prime
Best Case	$\theta(1)$
Worst Case	$\theta(n)$ but unlikely with good hash functions
Drawback	Slightly more complex implementation

Let me know if you'd like a **real C++ or Python implementation**, or examples of `hash1()` and `hash2()` functions!